

# Dynamic programming without the panic

Week 5, Lecture 1 · Landing the Offer: SWE Job Search for Senior CS Students

Autumn 2026

## What we will cover

- Why the recurrence relation comes before the code
- Memoization vs. tabulation: how to choose and how to convert
- 1D vs. 2D state: recognizing which shape a problem needs
- The four classic DP problems every candidate should know cold

By the end of this lecture you can write the recurrence for a 1D DP problem and convert it to a tabulated solution.

Recurrence first, code second

## The reason DP problems cause panic

- Most candidates open a DP problem and try to write loops immediately
- The loops do not make sense without knowing what the array cells represent
- Panic follows: the code accumulates, the logic does not
- Tushar Roy (2016): "Before writing a single line of code, draw out the recurrence relation. The code is just a translation of that table."

The fix is a forced pause: define the state, then write the recurrence, then write the code.

# What a recurrence relation is

A recurrence relation defines the value at position  $i$  in terms of earlier positions.

```
Climbing Stairs –  $dp[i] = dp[i-1] + dp[i-2]$ 
```

- $dp[i]$  = number of distinct ways to reach step  $i$
- Base cases:  $dp[0] = 1$ ,  $dp[1] = 1$
- Transition: to reach step  $i$ , you came from step  $i-1$  or step  $i-2$

Writing this before opening an editor catches wrong state definitions early.

## The three questions to answer before coding

1. **What does `dp[i]` represent?** One sentence. If you cannot write it, the state is not defined yet.
2. **What are the base cases?** The smallest valid inputs you can answer directly, without recursion.
3. **What is the transition?** How does `dp[i]` depend on `dp[i-1]`, `dp[i-2]`, or some earlier cell?

Only after answering all three do you open the editor.

# Memoization vs. tabulation

## Two ways to eliminate redundant computation

The naive recursive Fibonacci has  $O(2^n)$  time because it recomputes the same subproblems repeatedly.

Both approaches below fix this by storing results:

- **Memoization (top-down):** recurse normally; cache results in a dictionary or array before returning
- **Tabulation (bottom-up):** fill an array from small to large, reading earlier cells to compute each new one

NeetCode (2021): "Don't try to memorize solutions. Learn the pattern. Once you see the recurrence, you already know the code."



## Memoization in code

```
def climb_stairs(n: int) -> int:
    memo = {}

    def dp(i: int) -> int:
        if i <= 1:
            return 1
        if i in memo:
            return memo[i]
        memo[i] = dp(i - 1) + dp(i - 2)
        return memo[i]

    return dp(n)
```

Time:  $O(n)$ . Space:  $O(n)$  for the cache plus  $O(n)$  call stack.

## Tabulation in code

```
def climb_stairs(n: int) -> int:
    if n <= 1:
        return 1
    dp = [0] * (n + 1)
    dp[0] = 1
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]
```

Time:  $O(n)$ . Space:  $O(n)$ . Can be reduced to  $O(1)$  with two variables.

## How to choose between them in an interview

### Memoization

### Tabulation

- Easier to write from the recurrence directly
  - Call stack adds  $O(h)$  space
  - Natural fit when not all subproblems are needed
  - Can hit recursion depth limits on large  $n$
- 
- No recursion overhead
  - Space can often be compressed
  - Requires knowing the order you fill the table
  - Usually preferred when the interviewer

When in doubt, start with memoization to validate the logic, then mention you can tabulate it.

# 1D vs. 2D state

## 1D state: one variable drives the solution

When the answer at position  $i$  depends only on earlier positions in a single sequence, the state is 1D.

Common 1D DP problems:

- Climbing Stairs:  $dp[i]$  = ways to reach step  $i$
- House Robber:  $dp[i]$  = max money robbing up to house  $i$
- Coin Change:  $dp[i]$  = min coins to make amount  $i$

The array has one dimension matching the input sequence or the target value.

## 2D state: two variables drive the solution

When the answer depends on two changing inputs, you need a grid.

Longest Common Subsequence

`dp[i][j]` = LCS length of `s1[:i]` and `s2[:j]`

Base: `dp[0][j] = 0`, `dp[i][0] = 0`

Transition:

    if `s1[i-1] == s2[j-1]`: `dp[i][j] = dp[i-1][j-1] + 1`

    else: `dp[i][j] = max(dp[i-1][j], dp[i][j-1])`

The grid has rows for one input and columns for the other.

## Recognizing the state shape

Ask: how many things change from subproblem to subproblem?

- One thing changes (position in array, amount remaining): 1D
- Two things change (position in each of two strings, row and column in a grid): 2D
- Three things change: 3D (rare in interviews)

NeetCode roadmap covers 1D DP before 2D DP for this reason. Master the 1D shape first.

# The classic problem set



## Four problems to know before every interview

| Problem         | State                              | Transition                                |
|-----------------|------------------------------------|-------------------------------------------|
| Climbing Stairs | $dp[i]$ = ways to reach step $i$   | $dp[i-1] + dp[i-2]$                       |
| House Robber    | $dp[i]$ = max rob up to house $i$  | $\max(dp[i-1], dp[i-2] + \text{nums}[i])$ |
| Coin Change     | $dp[i]$ = min coins for amount $i$ | min over coins: $1 + dp[i - \text{coin}]$ |
| LCS             | $dp[i][j]$ = LCS of prefixes       | match: $dp[i-1][j-1] + 1$ ; no match: max |

(Coding Interview University, Washam 2016; NeetCode roadmap, 2023)

## House Robber: worked trace

Input: [2, 7, 9, 3, 1]

```
dp[0] = 2           (rob house 0)
dp[1] = max(2, 7) = 7
dp[2] = max(7, 2+9) = 11
dp[3] = max(11, 7+3) = 11
dp[4] = max(11, 11+1) = 12
```

Answer: 12 (rob houses 0, 2, 4 for 2+9+1).

State:  $dp[i]$  = max money using houses 0 through  $i$ .

Transition:  $dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$ .

## Take-aways

1. Write the recurrence before writing code. Define  $dp[i]$ , base cases, and transition in that order.
2. Memoization and tabulation produce identical results. Start with memoization to validate; tabulate to optimize.
3. Count how many variables change across subproblems. One variable means 1D state. Two means 2D.
4. The four classic problems (Climbing Stairs, House Robber, Coin Change, LCS) cover most 1D and 2D patterns you will see.
5. The panic comes from skipping the recurrence step. The recurrence is the solution.

"Before writing a single line of code, draw out the recurrence relation.  
The code is just a translation of that table."

Tushar Roy, Dynamic Programming playlist (2016)

## What's next

- **Reading:** Week 5 reading walks through House Robber end to end, from recurrence to tabulation
- **Lecture 2 (this week):** Communicating while you code
- **Section (this week):** Pattern sprint 3, two DP mediums plus a recorded narrated solve
- **HW2 (30-problem LeetCode sprint):** due this week
- **HW3 (recorded mock coding interview):** now out